



WORK INTEGRATED LEARNING PROGRAMMES

NLP Applications

Assignment – 2

Problem Statement – 9

PART A: Documentation

Group 55

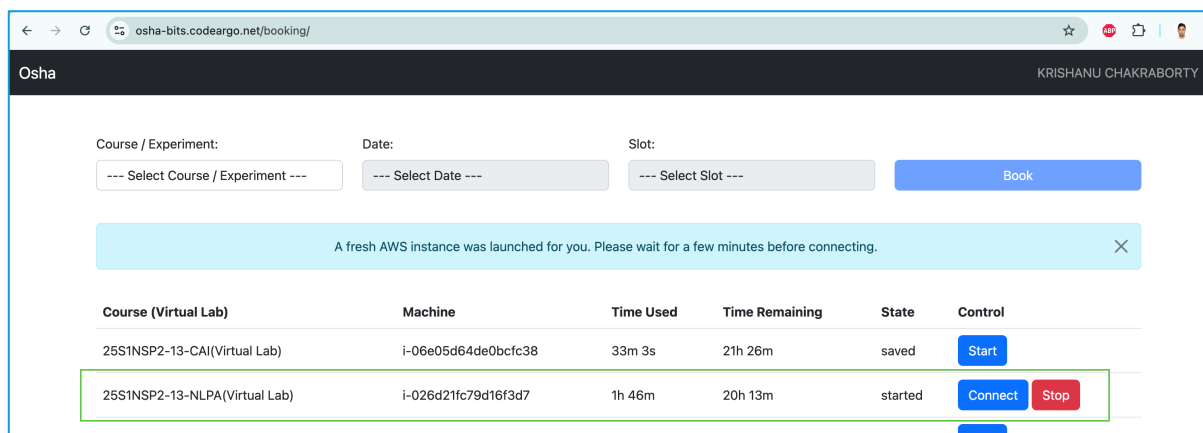
2024AA05192	Abhishek Kumar Tiwari
2024AA05193	Krishanu Chakraborty
2024AA05197	Viswanadha Pavan Kumar
2024AA05832	B Vinod Kumar
2024AB05168	K Abhinav

Table of Contents

.....	1
DEVELOPMENT INFRA USAGE	3
INSTALLATION GUIDE	5
BRIEF REPORT ON DESIGN CHOICE AND CHALLENGES	7
DESIGN CHOICE	7
CHALLENGES	8
COMPLETE APPLICATION FLOW	9
TASK-B: ENHANCEMENT PLAN.....	12
CONCLUSION.....	13

Development Infra Usage

We have used NLPA virtual lab in OSHA portal for the development and testing of the sentiment analysis application.

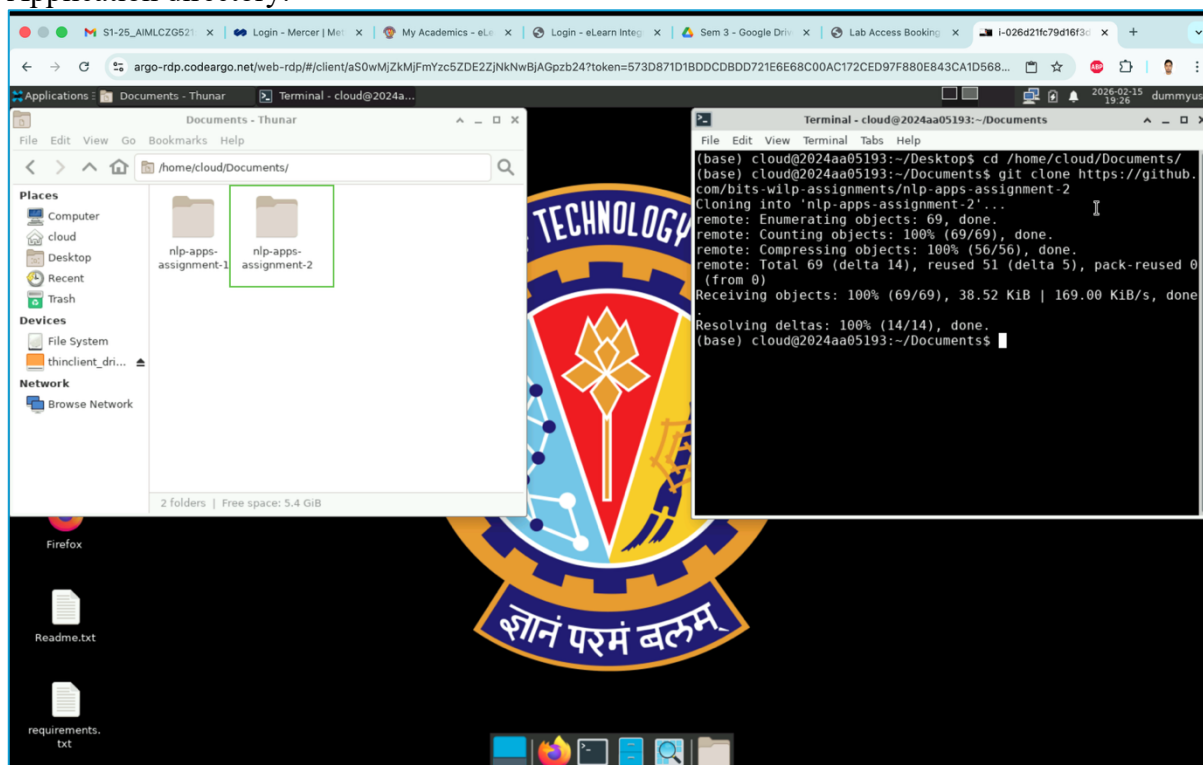


We have used GitHub to move the code developed from the local laptop to the virtual lab. We have created a **private** repo in GitHub to collaborate with the team members. We have made this repo as **public** for your access.

GitHub URL: <https://github.com/bits-wilp-assignments/nlp-apps-assignment-2>

Here's some screenshots of our activities in the virtual lab:

Application directory:



VS Code in virtual lab:

The screenshot shows a VS Code editor interface. The Explorer panel on the left displays the file structure of a project named 'NLP-APPS-ASSIGNMENT-2', including files like .streamlit, src, venv, .gitignore, README.md, requirements.txt, sentiment_app.py, and ui_app.py. The main editor window shows the code for ui_app.py, which imports Streamlit, requests, pandas, altair, and io, and defines a SentimentAnalysisUIApp class. The terminal window at the bottom shows the execution of 'python3 -m venv .venv', 'source venv/bin/activate', and 'pip install --upgrade pip', followed by the successful installation of pip 26.0.1.

```

1 import streamlit as st
2 import requests
3 import pandas as pd
4 import altair as alt
5 import io
6 from datetime import datetime
7 from src.config.ui_config import *
8 from src.util.logging_util import get_logger
9
10
11 class SentimentAnalysisUIApp:
12     """Streamlit UI application class for the Sentiment Analysis project.
13     This class sets up the Streamlit interface, handles user interactions, and communicates with the backend API f
14     """
15     def __init__(self):
16         self.logger = get_logger( name = )
17         self.api_url = BACKEND_BASE_URL
18         self._setup_page()
19
20     def _setup_page(self):
21         self.logger.info("Setting up Streamlit page configuration...")
22         st.set_page_config(
23             page_title=APP_TITLE,
24             page_icon="📊",

```

```

(base) cloud@2024aa95193:~/Documents/nlp-apps-assignment-2$ python3 -m venv .venv
(base) cloud@2024aa95193:~/Documents/nlp-apps-assignment-2$ source venv/bin/activate
(venv) (base) cloud@2024aa95193:~/Documents/nlp-apps-assignment-2$ pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.13/site-packages (25.2)
Collecting pip
  Downloading pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
  Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 83.7 MB/s 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.2
    Uninstalling pip-25.2:
      Successfully uninstalled pip-25.2

```

Installation Guide

Please follow the below steps for successfully setting up the **The Sentiment Analysis App**:

1. Get the latest code from GitHub repo <https://github.com/bits-wilp-assignments/nlp-apps-assignment-2> or refer provided the zip file to extract the code.
2. **Python 3.10** or greater version is required run this code. Use below command to verify the version from your terminal. Upgrade your python if required.

```
python --version
```

3. Make sure to change your directory where package file is extracted, verify by listing your directory, you should see the file app, run.py etc.
4. Create a Python virtual environment. Use the below commands:

```
python3 -m venv .venv  
source .venv/bin/activate
```

5. Once the Virtual environment python setup is completed , we need to get all the external libraries mentioned in requirements.txt. Use the below commands:

```
python3 -m pip install --upgrade pip setuptools wheel  
pip install -r requirements.txt
```

6. Install NLTK libraries for the virtual environment:

```
python3 -c "import nltk; nltk.download('punkt');  
nltk.download('punkt_tab'); nltk.download('stopwords');  
nltk.download('wordnet')"
```

7. After requirements installation, we can directly execute ui_app.py and sentiment_app.py to initiate the applications.

```
Python ui_app.py  
Python sentiment_app.py
```

8. Verify the application are up and running by executing below commands.

```
Python -version  
pip list | grep streamlit  
pip list | grep fast api
```

9. Test the app url API server on <http://localhost:8000>. It will display "Backend is ready!"
10. Test the streamlit UI URL APP server on <http://localhost:8501>

11. Click on this URL or copy paste in the browser
12. You should see “Application Homepage” as shown on the next page
13. Alternatively, the [README.md](#) from Github can be referred.
14. Refer the screenshots as well:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(venv) (base) cloud@2024aa05193:~/Documents/nlp-apps-assignment-2$ python sentiment app.py
2026-02-15 19:44:31 - src.core.preprocessing - INFO - SentimentDataPreprocessor initialized with data file: ['/home/cloud/.cache/kagglehub/datasets/saurabhshahane/twitter-sentiment-dataset/versions/1/Twitter Data.csv']
2026-02-15 19:44:31 - src.services.inference - INFO - LRPredictor initialized with model accuracy: 0.8576
2026-02-15 19:44:31 - __main__ - INFO - LRPredictor initialized successfully.
2026-02-15 19:44:31 - __main__ - INFO - Starting Sentiment Analysis API on 0.0.0.0:8000...
INFO: Started server process [6625]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:46596 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:46612 - "GET /config HTTP/1.1" 200 OK

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(venv) (base) cloud@2024aa05193:~/Documents/nlp-apps-assignment-2$ streamlit run ui app.py
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.31.109.74:8501

2026-02-15 19:45:19 - __main__ - INFO - Setting up Streamlit page configuration...
2026-02-15 19:45:19 - __main__ - INFO - Running Sentiment Analysis UI App...
2026-02-15 19:45:19 - __main__ - INFO - Checking backend status...
2026-02-15 19:45:19 - __main__ - INFO - Rendering Single Prediction tab...
2026-02-15 19:45:19 - __main__ - INFO - Rendering Batch Analysis tab...
2026-02-15 19:45:19 - __main__ - INFO - Rendering About tab...

```

Brief Report on Design Choice and Challenges

Design Choice

The application is designed as a minimalistic but realistic sentiment analysis application by combining a Python backend with a web based frontend powered by **Streamlit**.

- Loosely coupled backend to seaparate out data / logic from presentation, so the backend focuses on preprocessing of the text, training the model and using it for prediction while the frontend specializes in user interaction.
- Python **FastAPI** is used to host the API endpoints as backend:
 - Routes have been seaparted out from the actual business logic.
 - The API documentation (Swagger) is generated automatically as an additional advantage of using FastAPI.
 - **Services** contains the logic for live training during service initialization and inference of the Logistic Regression model (Scikit Learn).
 - The **core** directory contains the main python files including the actual implementation of preprocessing and training.
 - TF-IDF vectorizer has been used to convert the tokens into feature vectors which has been used by the Logistic Regression model to predict the probability of each class in order to ensure near-instant inference and low resource consumption.
 - The **util** directory contains the helper functions for logging and training data loading.
 - The config directory contains the constant configuration values for backend as well frontend completely separated from the core implementation and service layer.
- The below directory layout is used to seaprate out each layers distintly:

```

sentiment-analysis-app/
├── .streamlit/
│   └── config.toml          # Streamlit theme configuration
├── src/
│   ├── config/
│   │   ├── app_config.py    # Backend configuration
│   │   └── ui_config.py     # Frontend configuration
│   ├── core/
│   │   ├── preprocessing.py # Text preprocessing functions
│   │   └── training.py      # Model training logic
│   ├── services/
│   │   └── inference.py     # Prediction service
│   └── util/
│       ├── data_loader.py   # Dataset loading utilities
│       └── logging_util.py  # Logging configuration
├── sentiment_app.py         # FastAPI backend application
├── ui_app.py                # Streamlit frontend application
├── requirements.txt         # Python dependencies
└── README.md                # Project documentation

```

- The frontend component has been developed using Python **Streamlit** library for prototyping of our project and faster development.

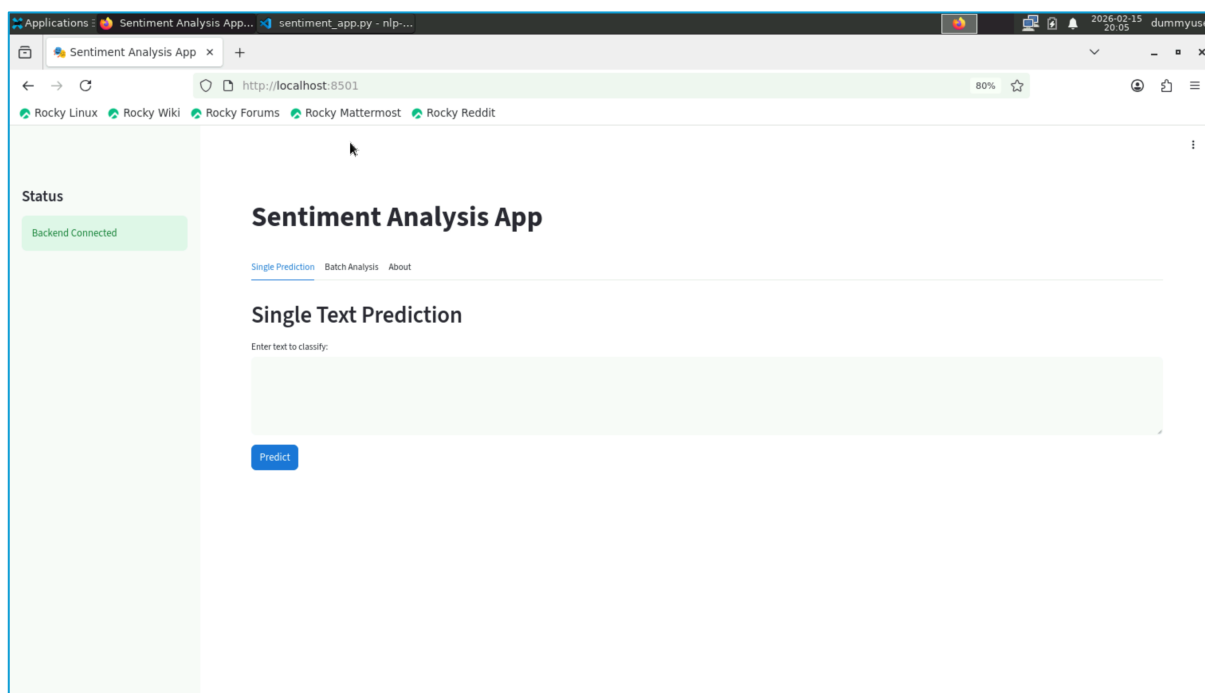
Challenges

Several challenges influenced these design choices:

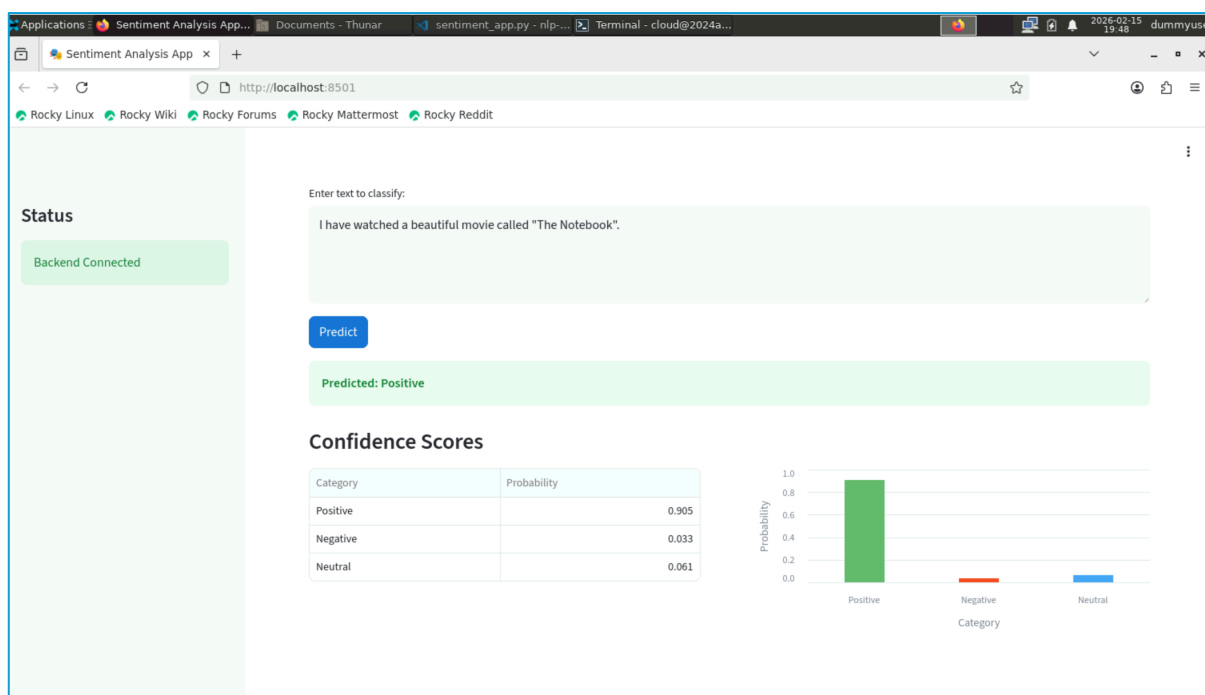
- **Balancing Model Performance with Real-Time Inference:**
 - While complex models offer higher accuracy, a Logistic Regression model with TF-IDF vectorization was chosen to ensure near-instantaneous sentiment prediction.
 - The core directory was designed to house the actual implementation of preprocessing and training to keep the pipeline optimized for these feature vectors.
 - The Services layer was implemented to manage the logic for live training during service initialization, ensuring the model is ready for inference immediately.
- **Architectural Decoupling for Maintainability:**
 - A loosely coupled backend was utilized to separate data and logic from the presentation layer, allowing the backend to focus strictly on text preprocessing and model training.
 - By using Python FastAPI, routes were separated from the actual business logic to ensure a modular structure that is easier to scale than a unified application.
 - Similar to avoiding complex JS frameworks like React for simple interactivity, Streamlit was selected to provide a realistic web-based frontend without adding unnecessary complexity to the tooling or state management.
- **Organization of Configuration and Utility Logic:**
 - A dedicated config directory was established to keep constant configuration values for the backend and frontend completely separated from the core implementation.
 - To prevent a cluttered codebase, the util directory was used to centralize helper functions for logging and training data loading.
 - Automatic API documentation (Swagger) was integrated as a built-in advantage of FastAPI to simplify the testing and validation of sentiment endpoints.

Complete Application Flow

1. Home Page – Once you finish Installation open localhost URL in browser
<http://localhost:8501>



2. We can navigate to different tabs like single prediction, batch analysis, about, and back end app status on the same home page.
3. Let's give "single prediction" as the first choice to test the application.
4. Ask a single prediction to the model and it will give the sentiment analysis and respect scores.



5. Lets create a Batch analysis by providing CSV file as input which is having multiple token inputs to analyse the sentiment and its scores.

Batch File Analysis

Upload CSV or Excel file

Drag and drop file here
Limit 200MB per file • CSV, XLSX

sentiment-texts.csv 96.08

Analyze Batch

Analyzed 3 comments

Total	Positive	Negative	Neutral
3	1	2	0

Top 10 Most Confident

text	predicted_category	max_prob
0 Beautiful movie, amazing acting.	Positive	99.6%
2 Waste of time	Negative	95.7%
1 Average acting, predictable plot	Negative	88.2%

Download Results

6. Let's list the existing configuration data and project overview by clicking about.

About This Application

Project Overview

This Sentiment Analysis application automatically classifies text into three sentiment categories: **Positive**, **Negative**, and **Neutral**. Built using machine learning techniques, it processes both single texts and batch files efficiently.

Model Information

Vectorization

- Method: TF-IDF (Term Frequency-Inverse Document Frequency)
- Max Features: 20,000
- N-gram Range: [1, 2]
- Stop Words: English

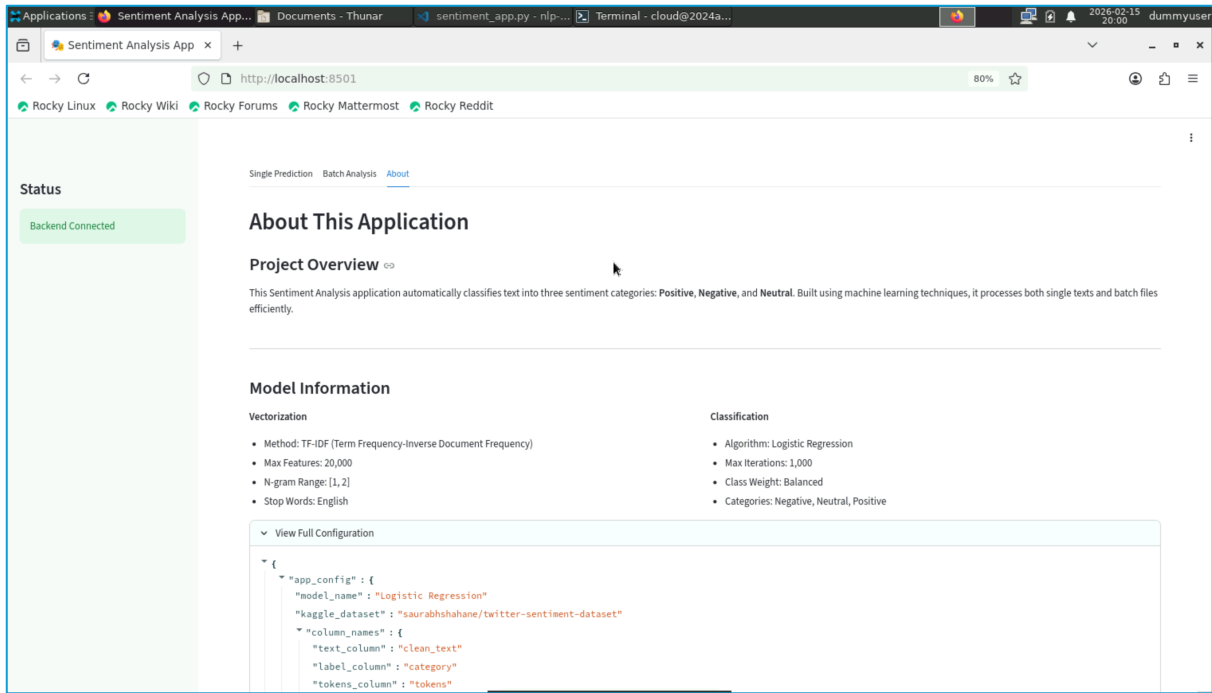
Classification

- Algorithm: Logistic Regression
- Max Iterations: 1,000
- Class Weight: Balanced
- Categories: Negative, Neutral, Positive

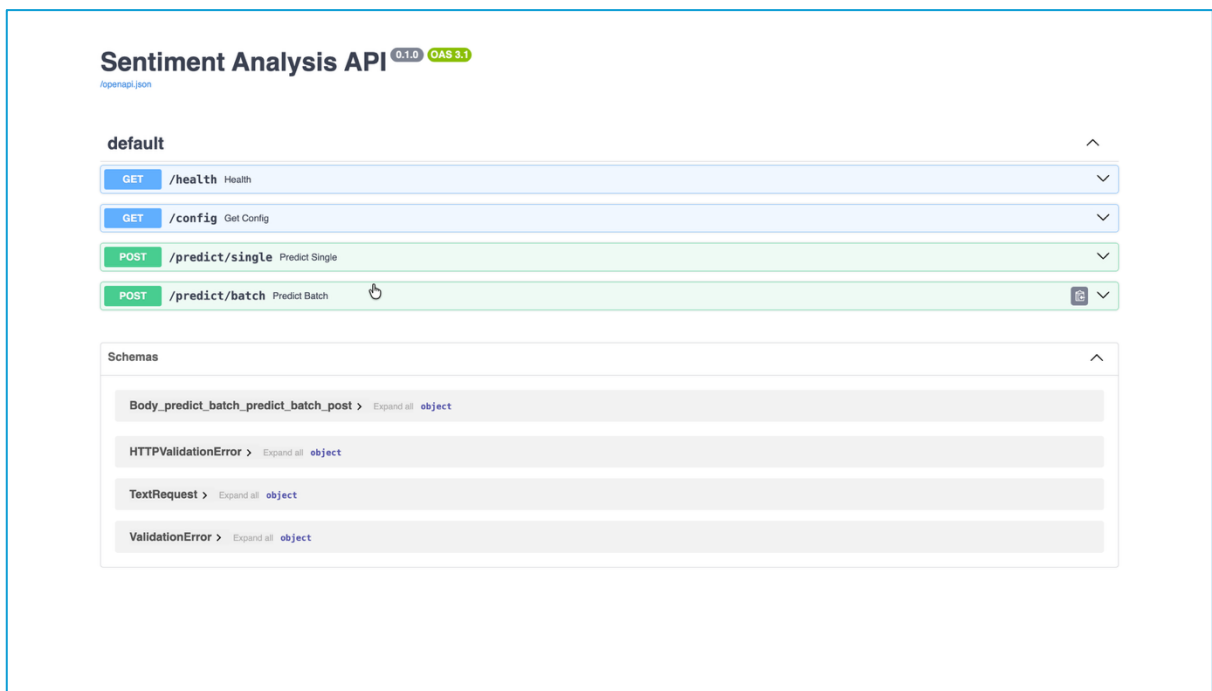
View Full Configuration

```
{
  "app_config": {
    "model_name": "Logistic Regression"
    "kaggle_dataset": "saurabhshahane/twitter-sentiment-dataset"
    "column_names": {
      "text_column": "clean_text"
      "label_column": "category"
      "tokens_column": "tokens"
    }
  }
}
```

7. How to use the tool and instructions are available in about page.



8. Additional information on API endpoint documentation:

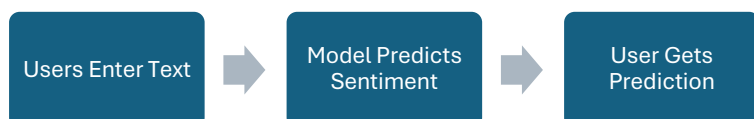


TASK-B: Enhancement Plan

The current Sentiment Analysis application classifies user-provided text into Positive, Negative, or Neutral categories using TF-IDF vectorization and a Logistic Regression model for multiclass prediction.

- While the system performs well on the trained dataset, it remains static after deployment. In real-world applications, language usage evolves over time, and model predictions may occasionally be incorrect. To address this limitation, a real-time feedback mechanism is proposed. This enhancement will allow users to provide immediate feedback on model predictions, enabling continuous improvement through retraining. The goal of this enhancement is to transform the application from a static prediction system into a closed-loop learning system.
 - User feedback capture
 - Feedback storage and validation
 - Feedback preprocessing
 - Periodic model retraining
 - Model evaluation and versioning

Current Process



Proposed Enhancement



- The text-based sentiment analysis can be enhanced further on the multimodal usecase
 - The plan integrates visual cues (facial expressions and gestures) and acoustic features (tone, pitch, and tempo) to capture emotional nuances like sarcasm or frustration that text alone often misses.
 - We could employ a late-fusion transformer architecture to cross-reference these modalities, ensuring the model prioritizes the most reliable data stream when signals conflict.
- Python **Streamlit** module has been used to develop the web based UI of the application. This is fine for prototyping, however, for production use case we need a robust JS based framework like **Angular** or **React** to develop the web based interface.

Conclusion

The integration of a real-time feedback mechanism transforms the existing sentiment analysis application into a dynamic and adaptive learning system. By capturing user corrections and incorporating them into periodic retraining cycles, the system continuously improves its predictive performance.

This enhancement ensures long-term model robustness, scalability, and alignment with real-world deployment standards.